

Week 2. Problem set (solutions by Karim Khabibrakhmanov)

1. In [CLRS, §16.1], a stack with an extra operation MULTIPOP is discussed. What is the total cost c of executing n of the stack operations PUSH, POP, and MULTIPOP, assuming that the stack begins with k objects? The answer must consist of exact lower and upper bounds given as formulae in terms of n , k (not asymptotic complexity!). Provide a brief justification (2–4 sentences). Formulas and equations without explanation will not be accepted.

Answer: $n \leq c \leq 2n + k - 2$

Justification:

Let's consider the cases for both the highest and lowest total cost:

1. The upper bound occurs when $n - 1$ **Push** operations are executed, each costing 1 unit, followed by a single **Multipop** that removes all elements from the stack. By that time, the stack contains $n - 1$ newly added elements plus the k initial elements. Summing the costs, we get:

$$(n - 1) + (n - 1 + k) = 2n + k - 2$$

$$c \leq 2n + k - 2$$

2. The lower bound for the total cost is at least n because we perform exactly n operations. If all of them are **Push** or **Pop**, each costs 1. Since k is unknown, we assume only **Push** operations, leading to the minimum cost:

$$n \leq c$$

3. Total cost of n operations:

$$n \leq c \leq 2n + k - 2$$

2. A sequence of PUSH, POP, and CLEAN operations is performed on an initially empty stack. When an element is first PUSHed to the stack, it is marked as **new**. The CLEAN operations inspects all elements of the stack:

- Every **new** element is marked as **old** (in constant time).
- Every **old** element is removed from the stack (in constant time).

Perform amortized time complexity analysis using the **accounting method** [CLRS, §16.2] for a sequence of PUSH, POP, and CLEAN operations performed on an initially empty stack:

- (a) Specify actual cost, amortized cost, and accumulated credit for each operation. Assume that n_i is the size of stack before operation and k_i is number of **old** elements in the stack.

Operation	Actual cost (c_i)	Amortized cost ($\hat{c}_i$)	Credit	Justification
PUSH	1	2	1	The actual cost of this operation is 1, and at the same time 1 credit is accumulated to cover the costs of future item removal operations
POP	1	0	-1	For this operation, the costs are covered by the credits that were initially allocated for the deletion
CLEAN	n_i	$n_i - k_i$	$-k_i$	The cost is covered by the credits for old elements k_i , and the remaining cost applies to elements transitioning from new to old, $n_i - k_i$.

- (b) Prove that the total amortized cost of a sequence of n operations provides an upper bound on the total actual cost of the sequence.

Proof. For each Push operation, we save 1 credit. For each Pop operation, there must have been a corresponding Push that covers its cost. For each old element in the Clean operation, we spend 1 credit, and for each new element in Clean, we incur a cost of 1, but this is not covered by credits.

Consequently, the total amortized cost is the upper bound for the total actual cost, since the loans cover the actual costs of Pop and Clean operations for the old elements, and the remaining cost for the new elements will not be spent from loans. $\square$

- (c) Write down the asymptotic complexity for a sequence of n operations.

Answer: $O(n)$

Derivation of the answer:

1. Each Push spends one actual $O(1)$ cost and an $O(1)$ removal cost
2. Each Pop operation is already covered in advance by a credit from the Push operation $O(0)$
3. Each Clean operation for old elements has already been closed in advance by a credit from the Push operation, and for new elements it is $O(1)$

Conclusion: Thus, the total amortized cost of a sequence of n operations is no more than $O(n)$.

3. Solve the previous exercise using the **potential method** [CLRS, §16.3]:

- (a) Define the potential function Φ on a stack; the potential function may depend on the size n of the stack and on the number k of **old** elements in the stack;

Answer: n

- (b) Compute $\Phi(D_i) - \Phi(D_{i-1})$ for each possible i th operation (PUSH, POP, CLEAN)

- PUSH: $\Phi(D_i) - \Phi(D_{i-1}) = (n_i + 1) - n_i = 1$
- POP: $\Phi(D_i) - \Phi(D_{i-1}) = (n_i - 1) - n_i = -1$
- CLEAN: $\Phi(D_i) - \Phi(D_{i-1}) = (n_i - k_i) - n_i = -k_i$

- (c) Compute amortized cost for CLEAN using your potential function;

Answer: $\hat{c}_i = c_i + (\Phi(D_i) - \Phi(D_{i-1})) = n_i + ((n_i - k_i) - n_i) = n_i - k_i$

- (d) Write down amortized asymptotic complexity for CLEAN.

Answer: $O(n)$

Derivation of the answer:

Since the Clean operation processes all n elements, its total cost is proportional to the number of elements, which is $O(n)$. Therefore, the amortized cost of this operation is also $O(n)$

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.